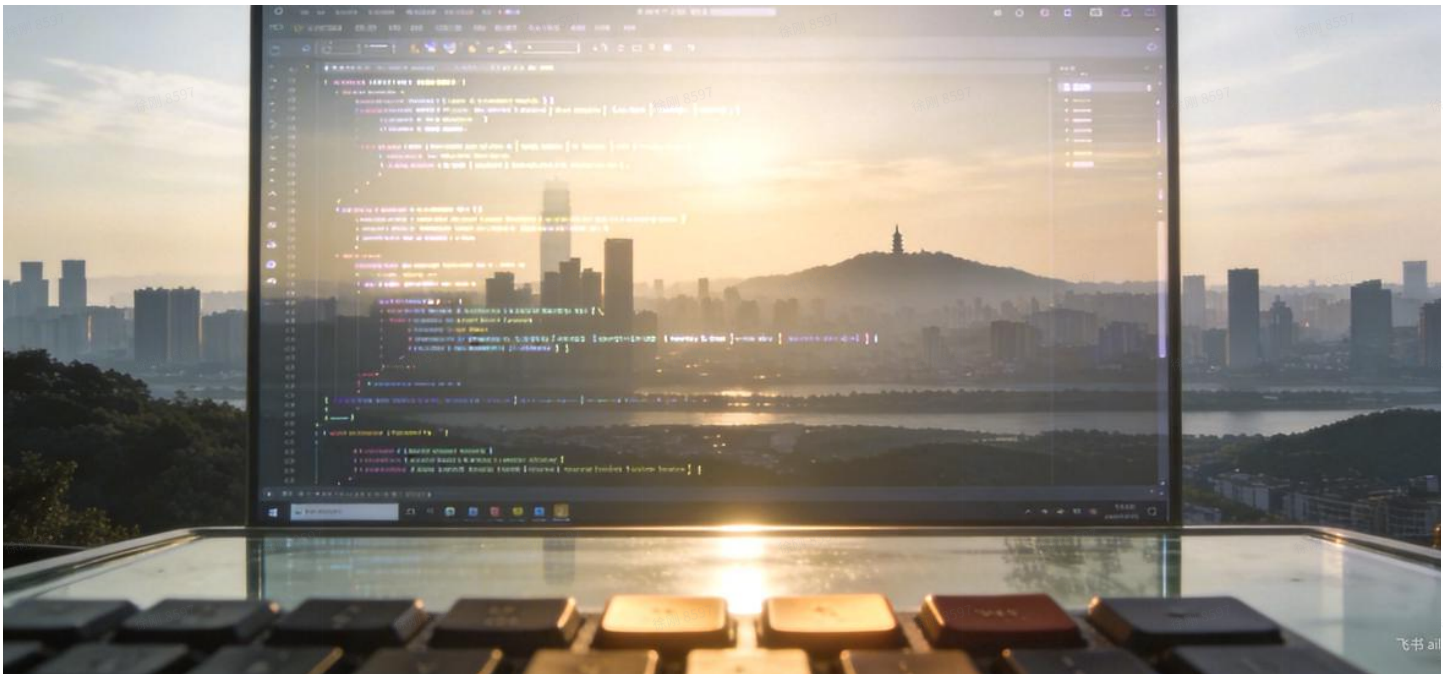

湖南CRM新业务开发AI Coding实战纪实：从闭源框架攻坚到研发范式升级

🎁 **破冰者的抉择：当公司"AI First"的战略号角吹响，湖南PSO作为事业部首批试点省份，面对的不是一片技术蓝海，而是SpringBoot+MyBatis+CSF+Wade(TouchUI)存量闭源框架**的"技术深水区"。转型于此，不仅是引入新工具，更是要攻克**框架老、规矩多、暗坑深**的历史技术债，系统性验证AI在真实、复杂研发环境中的可行性。这背后，是湖南人"**霸得蛮**"的韧劲与魄力，是为事业部规模化推广探路、沉淀可复用方法论的先锋使命。

一、启动背景：从战略响应到组织攻坚

2026年"丙午新春，亚信启新"的战略宣贯中，**AI优先**被确立为公司的核心生存法则与进化路径。对湖南PSO而言，这一战略并非遥不可及的蓝图，而是必须直面的现实挑战——**CRM订单中心**与**政企工程**两大核心业务系统，承载着繁重的业务工单与持续的迭代压力，传统"手工堆码"的开发模式在效率与质量上已触及瓶颈。湖南客户服务部成立**专项攻坚小组，在CMB AICoE**中心的支持下启动试点CRM新业务开发AI Coding新范式试点工作，试点工作摒弃了"大而全"的盲目铺开，确立了三大核心目标：验证AI在闭源框架下的**工程可行性**、沉淀可跨区域复用的**方法论与实践资产**、实现研发过程的**实质性提质增效**。

二、场景破局：构建AI Coding四步法实战体系

转型的真正落地，在于业务场景中的**闭环价值**。我们摒弃散点尝试，针对四大高频痛点场景，构建了"认知-规范-执行-校验"的**四步法实战体系**，将AI从"工具尝鲜"转变为可量化、可复用的生产力。

方法论核心（左：如何做）

- **是什么？——知识沉淀**：将AI无法自主获取的**项目专有知识**（技术栈、架构、数据路由）体系化，弥合认知鸿沟。
- **怎么做？——规范定义**：用"正例引导+反例约束+决策规则+清单兜底"四重保障，将团队编码规范转化为AI可执行的生成规则。
- **做什么？——技能封装**：识别高频、模式固定的开发场景，封装为可一键触发的标准化**Skill**，如 `task_table_to_crud` 。
- **好不好？——质量校验**：通过"生成质量验证→规则溯源分析→规则迭代优化"闭环，驱动体系持续进化。

实战成果（右：做得怎样）

- **DDL转CRUD全栈**：输入表名，自动生成横跨SVC、API、WEB层的8+文件，处理14种数据源映射，**综合提效78%**。
- **Plus插件开发**：基于@Plus注解体系，自动生成订单流程业务插件代码及单元测试，**提效约66.7%**。
- **Rule规则生成**：从30+种Fact对象中精准匹配，自动声明@ATTR属性，消除"运行时null"陷阱，**提效62.5%**。
- **政企全栈生成**：支持Bean/SVC/Controller后端与Wade前端全流程生成，内置Import防遗漏机制，**提效75%**。

量化效能与资产沉淀

试点场景的效能提升并非主观感受，而是通过严格的**耗时评估与效率分析**得出的量化结论：

场景	传统模式耗时	AI新范式耗时	效率提升	关键价值点
DDL转CRUD全栈	9小时	2小时	**78%**	一键生成8文件，自动处理多数据源路由
Plus插件开发	4.5小时	1.5小时	**66.7%**	注解配置自动化，单元测试同步生成
Rule规则生成	4小时	1.5小时	**62.5%**	Fact智能匹配，杜绝属性声明遗漏
政企全栈开发	8小时	2小时	**75%**	双模式支持，Import零遗漏

在实战中，我们沉淀了可直接驱动AI的**标准化Skill库**与**质量验收Checklist**，确保产出质量可控：

Skill	类型	输入	输出	质量审查要点
`task_table_to_crud`	生成型	DDL建表语句	后端全层代码+前端页面	目录正确性、数据源注解、字段映射

`task_plus_dev`	生成型	业务需求描述	插件代码+单元测试	@Plus注解参数、台账操作规范
`task_rule_dev`	生成型	业务需求描述	规则代码+单元测试	@ATTR属性完整性、Fact类型匹配
`task_param_dev`	生成型	配置需求描述	SQL配置脚本	编码唯一性、多库脚本分离

三、实战案例深度剖析：从全栈CRUD到业务插件自动化

方法论的价值需要通过具体场景验证。湖南团队选取了两个在复杂度、业务价值上具有代表性的场景进行深度试点，形成了鲜明的对比与互补。

维度	**案例一：DDL建表语句转CRUD全栈代码**	**案例二：Plus订单插件自动化开发**
场景类型	全栈代码生成（后端+前端）	业务逻辑插件开发
技术难点	跨3个工程层（SVC/API/WEB）的规范统一；14种数据库用户的数据源自动路由与配置。	复杂的**注解体系**（@Plus参数）；订单**上下文（Context）** 数据的正确获取与使用；严禁直接调用OpenService。
AI核心价值	**一键全栈**：输入表名，自动完成表结构解析、全层代码生成、前端页面搭建，处理多源路由。	**精准匹配**：根据业务描述自动匹配正确的**Fact对象**，生成完整的**@ATTR属性声明**，从源头消除“运行时null”问题。
提效数据	综合提效 **78%**（9小时→2小时）	综合提效 **66.7%**（4.5小时→1.5小时）
推广价值	**自动化程度最高**，模式固定，是规模化推广的首选场景，能快速释放基础开发人力。	**质量保障最强**，将资深开发者的框架知识转化为AI规则，大幅降低新人学习成本与出错率。

案例一 选择了“DDL建表语句→全栈CRUD代码”作为首发场景，因其覆盖面最广、模式高度重复。传统开发中，新建数据表后的CRUD代码编写涉及Entity字段映射、Mapper XML编写、Service标准方法等8+个文件，人工编写费时且易遗漏细节。AI在此场景实现了**8步流水线**自动化，同步处理了地市路由、用户ID路由、分表策略等多种配置，并将前端页面生成作为可选步骤，展现了强大的全栈集成能力。

案例二 则深入订单流程的**Plus插件开发**，这是业务逻辑的核心载体。其难点不在于代码量，而在于对复杂框架规范的精准遵循。团队通过 **“五步法”**（需求分析→框架确认→AI生成与审核→测试验证→规则迭代）进行严格管控。AI的价值体现在自动生成符合 **@Plus** 注解规范、正确使用 **Context** 获取数据、通过**Callout封装类**进行服务调用的高质量代码，将开发者从繁琐的规范记忆中解放出来，专注于业务逻辑本身。

两个案例共同表明，**AI Coding的深层价值不仅是“写得快”，更是“写得好”**。通过将规范内置到生成环节，AI从根本上消除了人为疏忽导致的高频低级错误，为研发质量树立了新的底线。

四、认知跃迁：从个人编码提效到组织知识资产沉淀

试点最深刻的洞察，是彻底击碎了"**AI=自动写代码**"的浅层认知。我们意识到，AI生成代码的准确性与深度，**高度依赖于企业存量框架知识与业务知识的沉淀质量**。代码只是载体，而将隐性知识显性化、结构化，构建组织的"数字基因"，才是这场转型的核心战略价值。

1、知识资产库：从误判到体系构建

初期我们误以为AI出错是随机的，试图通过"重试"解决，结果走了弯路。后来发现，问题有规律可循（如DatasetList Import遗漏、@ATTR属性不全），策略遂转变为"**建规则**"——将防线前置到知识沉淀环节。经过持续迭代，我们为两大工程系统构建了规模可观的知识规则库：

工程系统	规则库层级	代表文件	核心内容摘要	行数
CRM订单中心	架构设计层	`common_rule.md`	整体架构、技术栈、分层开发、数据库路由、事务日志	486
	技术规范层	`rule_ddl_to_java.md`	DDL解析、Entity→OpenService各层代码生成规范	632
	专项规范层	`rule_plus_dev.md`	@Plus注解、PlusType、Direction、返销处理规范	163
	小计	**8个文件**		**2661行**
政企工程	架构设计层	`project_rule.md`	项目架构、IData/IDataset数据对象、Dao API规范	1150
	技术规范层			432

		`project_rule_crud_template.md`	Bean/SVC/Controller三层CRUD完整模板	
	专项规范层	`offer_config_rule.md`	资费包配置9表11步流程、uop_upc/uop_sys分库处理	521
	小计	**12个文件**		**7243行**

这两大知识库，总计20个文件，9904行规范，构成了湖南PSO在AI Coding转型中最宝贵的数字资产。

2、开发范式的根本性转变

AI Coding重塑了开发团队的工作模式与能力要求：

- **角色重塑**：开发者从"代码创作者"转变为"**需求定义者**"与"**架构审查者**"。核心工作前移至精准描述需求、定义约束条件，后移至高质量代码审查。
- **知识传承升级**：传统依赖"老员工口口相传"的隐性知识（如14种数据源映射关系），现被**规则文件+AI工具**固化，新人学习曲线显著降低。
- **能力考验加剧**：AI处理了重复劳动，留下的都是需要深度思考的复杂问题。**不懂业务、不熟架构的人，使用AI反而更容易产出隐蔽错误**。团队必须构建"T型深度+π型广度"的复合能力模型。

> 🎁 **核心教训**："编译通过"远不等于"逻辑正确"。最危险的代码是那些语法无误但业务逻辑偏差的AI产出。例如，Rule规则中@ATTR遗漏一个属性，会导致该属性运行时返回null，进而引发业务判断全线错误，且系统不会报错。**必须建立并严格执行逐行审查机制，将质量底线从编译器前移至人脑。**

五、实战心得体会

坦率地说，团队最初对AI Coding的期待和实际落地之间有不小的落差。刚开始大家觉得"AI写代码"应该是输入需求、输出成品、直接可用，但真正上手后才发现远没有那么简单。CRM订单中心和政企业务系统都是跑了多年的老系统，框架老、规矩多、暗坑深，AI对这些"老框架"和"历史代码"的理解几乎为零。

前两周基本在踩坑，AI生成的Plus插件代码，PlusType写对了但Direction搞反了；政企工程也是错误频发，Bean层代码看着没问题，跑起来才发现少导了包或者生成代码Bean与SVC层返回类型不一致，或者对DatesetList用法出了偏差。

经过不断地调试，终于在第三周有了好转。我们把踩过的坑系统地写成规则文件，告诉AI"这里该注意什么、那里该怎么写"。规则文件从最初几十行的草稿，逐渐长成了现在CRM 8个文件2661行、政企12

个文件7243行的规模。说实话，整理这些规则比用AI写代码本身还累，但效果立竿见影。加上规则约束后，AI的生成质量明显上了一个台阶。

1、作业模式的变化

最直观的感受是开发者的日常工作内容变了。

以前写一个CRM的CRUD功能，从DDL建表开始，Entity、Mapper、Service、DTO、OpenService五层Java代码要一个个手写，再加上HTML、Page、JS三个前端文件，还有菜单注册和页面注册，全流程下来基本上一天时间过去了。政企那边的全栈开发也差不多，Bean、SVC、Controller三层后端加上前端五个文件，做完一套标准CRUD功能基本上也要大半天。

现在这些重复性的编码工作基本交给了AI。开发者花更多时间在两件事上：

一是前期把需求说清楚：用哪个数据源、走什么路由策略、字段映射关系是什么；

二是后期审查AI出的代码，如Import全不全、注解对不对、分层规范有没有越界。

编码环节确实效率提升了很多，但代码审查环节比原来预想的要花时间。

另一个没预料到的变化是知识传承的方式。以前新人来了全靠老员工带，CRM的14种数据源映射关系、政企的IData和IDataset该怎么用、服务怎么注册等等，这些东西文档里写得粗，细节全在老人脑子里。现在这些经验被逼着写成了规则文件，新人拿着规则文件配合AI工具，上手速度确实快了不少。

2、对团队能力的新考验

试点中有一个现象值得关注：同样的工具和规则文件，不同人用出来的效果差很多。

首先是需求描述精准，比如做Rule开发时，会明确说"基于TradeUser这个Fact对象，需要校验USER_ID和EPARCHY_CODE"，而不是笼统地说"写个用户校验规则"。

其次是对生成结果有"直觉"，一眼扫过去就能感觉哪里不对，这种直觉背后其实是对系统架构的深度理解。

说到底，AI把简单的活干了，留下来的都是需要动脑子的活。不懂业务、不熟架构的人，用了AI反而可能比以前更容易出错。因为以前手写代码的过程本身就是一种检查，现在跳过了这个过程，如果审查再不到位，问题会藏得更深。

3、风险与教训

3.1 我们对AI写代码这件事的几个误判

回过头来看，团队在试点初期对AI Coding的认知有几个偏差，这些偏差直接导致了我们走了一些弯路。

误判一：以为AI出错是随机的，其实是有规律可循的。

刚开始AI生成的代码有问题，大家的反应是"AI不靠谱，再生成一次试试"。后来慢慢发现不对，同样的问题它会反复犯。政企模块的DatasetList Import遗漏就是这样，不管提示词怎么换，它大概率还是会

漏。CRM这边，Rule开发里@ATTR属性不全的问题也是，如果你不在规则文件里明确列出来，它下次还是会少。

意识到这一点之后，我们的策略从"重试"变成了"建规则"。与其每次盯着AI改，不如搞清楚它在哪些地方容易犯错，把防线前置到规则文件里。现在来看，这个认知转变大概是整个试点过程中最重要的一步。

误判二：以为代码能编译通过就算合格，其实最危险的恰恰是"能跑但不对"的代码。

这是一个差点造成严重后果的教训。AI生成的代码语法层面基本没问题，编译100%能过，但"编译通过"和"逻辑正确"之间的距离比我们想象的远得多。

举两个真实案例：CRM的Rule引擎里，@ATTR注解少声明了一个属性，代码编译正常、启动正常，调用的时候这个属性返回null，后面的业务判断全走了默认分支——系统没报错，但处理结果是错的。

另一个在参数配置场景，AI生成的SQL脚本把两个不同库的脚本生成到了一个文件里，脚本执行的时候就会报错。

这类问题用自动化手段很难发现，只有熟悉业务的人仔细逐行审查才能抓到。后来我们在每个场景的规则文件中增加审查Checklist，把这类容易被忽视的检查项单独标出来。

误判三：以为规则文件写一次就够了，其实它需要持续迭代。

最初整理规则文件的时候，大家觉得把框架规范和编码约定写清楚就差不多了。但实际用下来发现，真正决定生成质量的往往是那些很细碎的"注意事项"——CRM这边，比如哪种Plus类型对应哪种Direction组合、Databus里哪些字段是必传的；政企那边，比如IDataset取值要用getString而不是get、页面注册文件的路径拼写规则。

这些东西太细了，第一版规则根本不可能全覆盖，只有在实际使用中踩了坑才会想到补进去。所以规则文件本质上不是一份"文档"，更像一个在持续生长的知识库。每发现一个新的高频问题，就加一条规则或案例，AI的生成质量就上一个台阶。

3.2 哪些风险是可控的，哪些到现在仍然棘手

经过几轮迭代，有些风险已经被压得比较低了。像Import遗漏、注解不全、注册配置缺失这类问题，通过Checklist强制检查基本能兜住。只要审查流程执行到位，出问题的概率很小。

但有些风险到现在还没有特别好的办法：

存量代码的适配冲突。 CRM老的Plus插件有些写法跟规则文件定义的规范不一样，新代码严格按规范来，放在老代码堆里显得格格不入。改老代码动静太大，不改又风格混乱。政企那边因为是长流程更复杂，AI对它的理解完全依赖我们投喂的规则，一旦遇到规则没覆盖的场景，生成结果就不太可控。

复杂业务逻辑的处理。 标准CRUD和模板化场景效果很好，但一涉及到业务效果就不行了。比如CRM的订单受理流程涉及多个工位之间的数据流转，Plus插件要在特定工位节点插入逻辑，这种跨环节的业务理解AI目前做不了，只能靠开发人员手动处理。

隐性知识的挖掘成本。 参数配置场景的编码规则是最好的例子。如资费编码以84打头、产品ID以1100开头，这些规则没有任何正式文档记录，全是从老员工那里一条条问出来的。这类隐性知识的规则化成本很高，而且无法确定是不是还有什么没问到的或者当时没想到的。

六、后续展望

本次试点所覆盖的DDL转CRUD全栈、Plus插件开发、Rule规则生成、政企全栈开发四类场景，相较于CRM新业务开发的整体需求而言，仍只是很小的一部分。CRM订单中心与政企工程历经多年建设，存量复杂业务场景众多、业务规则差异显著，本次试点仅完成了"万里长征第一步"的探索验证。若要实现AI Coding在存量复杂工程新业务开发领域的规模化应用，还需要在更多场景的规则沉淀、跨区域方法论推广及复合型人才培养等方面持续投入资源，以支撑这一转型目标的达成。

(注：内容由 AI 生成，请谨慎参考)